HOSTEL PG TRACKER

CAPABILITES

1. User Authentication — Register and log in as a student or PG owner.

2. Search Accommodations — Search for hostels and PGs based on location, rent, and preferences.

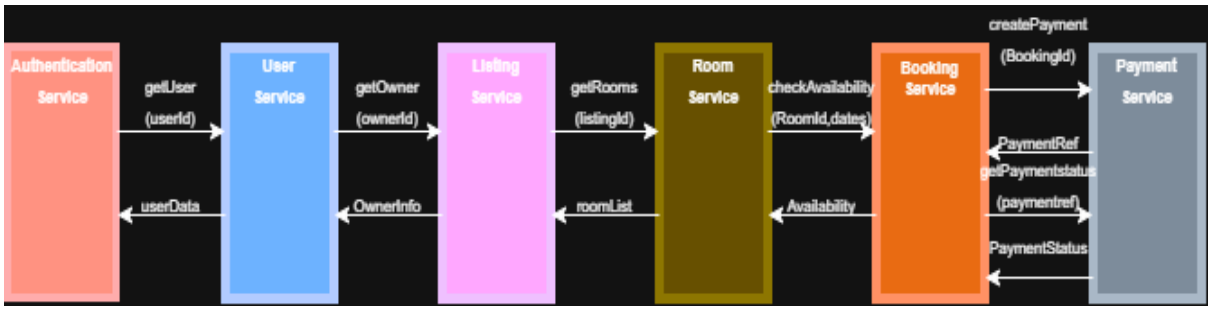
3. Manage Listings — Allow PG owners to create, update, and delete accommodation listings.

4. Check Availability — View available rooms and beds in hostels and PGs.

5. Manage Bookings — Create, view, and cancel accommodation bookings.

6. Process Payments — Make booking-related payments and check their status.

SERVICE DIAGRAM



CONTRACTS

1. Listing Service — Main Service

Purpose: Manages PG listings and provides accommodation information to other services.

Operation name	Input	Output	Errors
searchPGs()	Location, optional rent range and filters	Matching PG summaries	INVALID_FILTER, SERVICE_UNAVAILABLE
getListingDetails()	Listing reference	Listing details, status, and amenities	LISTING_NOT_FOUND, LISTING_UNAVAILABLE
createListing()	Owner reference, listing details	Listing reference and creation confirmation	INVALID_INPUT, OWNER_NOT_ELIGIBLE
updateListing()	Listing reference, owner reference, updated details	Update confirmation	LISTING_NOT_FOUND, FORBIDDEN, INVALID_INPUT

2. Booking Service — Main Service

Purpose: Manages accommodation booking requests, cancellations, and booking status.

Operation name	Input	Output	Errors
createBooking()	Student reference, listing reference, room reference, stay period	Booking confirmation and status	INVALID_INPUT, LISTING_UNAVAILABLE, NO_AVAILABILITY
getBookingDetails()	Booking reference, requesting user	Booking details and status	BOOKING_NOT_FOUND, FORBIDDEN
cancelBooking()	Booking reference, requesting user, cancellation reason	Cancellation confirmation and status	BOOKING_NOT_FOUND, FORBIDDEN, CANCELLATION_NOT_ALLOWED
getBookingHistory()	User reference, optional filters	List of relevant bookings	FORBIDDEN, INVALID_FILTER

3. Payment Service — Supporting Business Service

Purpose: Handles payment initiation, verification, and payment status.

Operation name	Input	Output	Errors
initiatePayment()	Booking reference, amount, currency	Payment reference and payment status	INVALID_AMOUNT, BOOKING_NOT_FOUND, PAYMENT_INITIATION_FAILED
verifyPayment()	Payment reference, verification evidence	Verified payment result and status	PAYMENT_NOT_FOUND, VERIFICATION_FAILED
getPaymentStatus()	Payment reference	Current payment status and amount	PAYMENT_NOT_FOUND, SERVICE_UNAVAILABLE
refundPayment()	Payment reference, refund amount, reason	Refund reference and refund status	PAYMENT_NOT_FOUND, REFUND_NOT_ALLOWED, REFUND_FAILED

4. Room Service — Supporting Service

Purpose: Manages rooms, beds, and accommodation availability.

Operation name	Input	Output	Errors
getRooms()	Listing reference	Room details and availability	LISTING_NOT_FOUND, SERVICE_UNAVAILABLE
checkAvailability()	Room reference, requested stay period	Availability result	ROOM_NOT_FOUND, INVALID_DATES
reserveBed()	Room reference, booking reference, stay period	Reservation confirmation	ROOM_NOT_FOUND, NO_AVAILABILITY, RESERVATION_CONFLICT
releaseBed()	Reservation reference, reason	Release confirmation	RESERVATION_NOT_FOUND, RESERVATION_ALREADY_RELEASED

5. Authentication Service — Supporting Service

Purpose: Authenticates users and verifies their identity.

Operation name	Input	Output	Errors
register()	Name, email, password, role	Registration confirmation and user reference	INVALID_INPUT, EMAIL_ALREADY_EXISTS
login()	Email, password	Authentication token and expiry	INVALID_CREDENTIALS, ACCOUNT_DISABLED
verifyToken()	Authentication token	Validity, user reference, and role	INVALID_TOKEN, TOKEN_EXPIRED

6. User Service — Supporting Service

Purpose: Manages user profiles and account information.

Operation name	Input	Output	Errors
getUser()	User reference	User profile and account status	USER_NOT_FOUND, SERVICE_UNAVAILABLE
getOwner()	Owner reference	Public owner profile and eligibility	OWNER_NOT_FOUND, OWNER_NOT_ELIGIBLE
updateProfile()	User reference, updated profile details	Updated profile confirmation	USER_NOT_FOUND, INVALID_INPUT

Hostel PG Tracker: Evaluation of the Six Microservices

A service should satisfy five properties: reachable, self-contained, has a contract, independent, and loosely coupled. Based on your six services and their contracts, here is an evaluation suitable for your software engineering assignment.

1. Listing Service — Main Service

Property	Status	Reason
Reachable	Pass*	Other services can call its operations through an API.
Self-contained	Pass*	It manages PG listings and accommodation information in its own database.
Has a contract	Pass	searchPGs(), getListingDetails(), createListing(), and updateListing() define inputs, outputs, and errors.
Independent	Partial	It depends on User Service to validate the owner and eligibility.
Loosely coupled	Partial	Other services depend on listing references and availability information.

Fix: Expose stable APIs, own the listing database, and use owner_id as an external reference rather than a cross-database foreign key. Define clear timeout and error-handling behavior for User Service calls.

2. Booking Service — Main Service

Property	Status	Reason
Reachable	Pass*	Students can invoke booking operations through an API.
Self-contained	Partial	It owns booking records, but completing a booking requires Listing and Room Services.
Has a contract	Pass	The four operations specify inputs, outputs, and error cases.
Independent	Partial	Booking creation depends on listing validation and room reservation.
Loosely coupled	Partial	It communicates with multiple services and relies on their availability.

Fix: Maintain a separate booking_db, communicate through documented APIs, and use a reservation workflow with compensation. For example, release the reserved bed if saving the booking fails. Use idempotency to prevent duplicate bookings after retries.

3. Payment Service — Supporting Business Service

Property	Status	Reason
Reachable	Pass*	Payment operations can be exposed through an API.
Self-contained	Pass*	It can own payment transactions and refund records.
Has a contract	Pass	initiatePayment(), verifyPayment(), getPaymentStatus(), and refundPayment() are defined.
Independent	Partial	It needs booking information and an external payment gateway.
Loosely coupled	Partial	Payment processing depends on Booking Service and the payment provider.

Fix: Keep payment data in payment_db, use booking_id as a reference rather than a cross-service foreign key, and verify payment status using trusted gateway responses. Make payment requests idempotent and handle failed transactions safely.

4. Room Service — Supporting Service

Property	Status	Reason
Reachable	Pass*	It exposes room and availability operations.
Self-contained	Pass*	It owns room, bed, and reservation data.
Has a contract	Pass	Its four operations define inputs, outputs, and errors.
Independent	Partial	It needs listing references and must coordinate reservations with Booking Service.
Loosely coupled	Partial	It depends on listing identifiers and reservation requests from Booking Service.

Fix: Keep rooms, beds, and reservations in room_db. Use external listing IDs without cross-database foreign keys. Implement atomic bed reservation so simultaneous requests cannot reserve the same bed.

5. Authentication Service — Supporting Service

Property	Status	Reason
Reachable	Pass*	Registration, login, and token verification are exposed as operations.
Self-contained	Partial	It may depend on User Service during registration to create a user profile.
Has a contract	Pass	Its three operations define inputs, outputs, and error cases.
Independent	Partial	Registration and identity management may require coordination with User Service.
Loosely coupled	Partial	It needs a clear agreement with User Service about user IDs and registration failures.

Fix: Keep credentials, password hashes, and authentication tokens in auth_db. Keep profile data in user_db. Define a registration workflow that handles failures between the two services, and use standard token validation and secure password hashing.

6. User Service — Supporting Service

Property	Status	Reason
Reachable	Pass*	Other services can call profile and owner operations through APIs.
Self-contained	Pass*	It owns profile and owner information.
Has a contract	Pass	getUser(), getOwner(), and updateProfile() specify inputs, outputs, and errors.
Independent	Partial	Owner eligibility may depend on verification processes or account status.
Loosely coupled	Partial	Listing and Authentication Services depend on user references and eligibility information.

Fix: Maintain profile data in user_db, expose stable profile and eligibility APIs, and avoid sharing database tables with other services. Define which service is authoritative for account status and owner verification.